

1. Declaration / Prototyping the function
2. Call of a function
3. Definition

.h

//Prototype

<return type> name of the Function (<arguments>);

int add(int,int);

void main()

{

 //Call

 int result = add(4,5);

}

//Definition

int add(int num1, int num2)

{

 return num1 + num2;

}

.libraries

stdio.h - stdio.lib

.c -> Compiler -> M/c code (.obj)

directly execute the .obj

Linker/Assembler

1. Creating .exe file based on COFF

2. Assembling/Linking all libraries to the current binary object file created for my code

```
#include<stdio.h>
```

1. Scope of Variable

2. Life of Variable

```
void main()
```

```
{
```

```
    int num1;
```

```
    int * ptr = display();
```

```
    *ptr ?
```

```
}
```

```
int * display()
```

```
{
```

```
    int data = 11;
```

```
    printf("%d",data);
```

```
    return &data;
```

```
}
```

Pointer : Is a variable that holds the memory address of another variable.

Size of int pointer in C : 2 byte

32 bit : 4 byte

// 32 bit arch

size of long in C : 4

flota : 4

int : 4

double : 8

char : 1

int *ptr; = 4

float * = 4

long * = 4

char * = 4

double* = 4

struct Data * = 4

struct Data

{

int value; 4

int age; 4

double salary; 8

}

1. Stack / Local Store
2. Heap / Free Store
3. Global / Static Store

